



Operating system and system programming (25CS2104E)

Linux Concurrent Producer–Consumer System Using Semaphores

Review-I

Under the Esteemed Guidance of

Raghupathi M
CSE Department

Team Members:

M. Laasya sri	2520090196
M. Rajini	2520090197
K. Ramya	2520090213

Contents

- Abstract
- Introduction
- Hardware and Software requirements
- System Architecture
- Conclusion

The **Producer–Consumer problem** is a classic Operating System problem that demonstrates **process synchronization and concurrent execution**.

In this project, a Linux-based Producer–Consumer system is developed using **semaphores** to coordinate producer and consumer processes. The producer generates data and places it into a shared buffer, while the consumer retrieves and processes the data.

Semaphores are used to prevent:

- Race conditions
- Buffer overflow
- Buffer underflow
- Unsafe access to shared resources

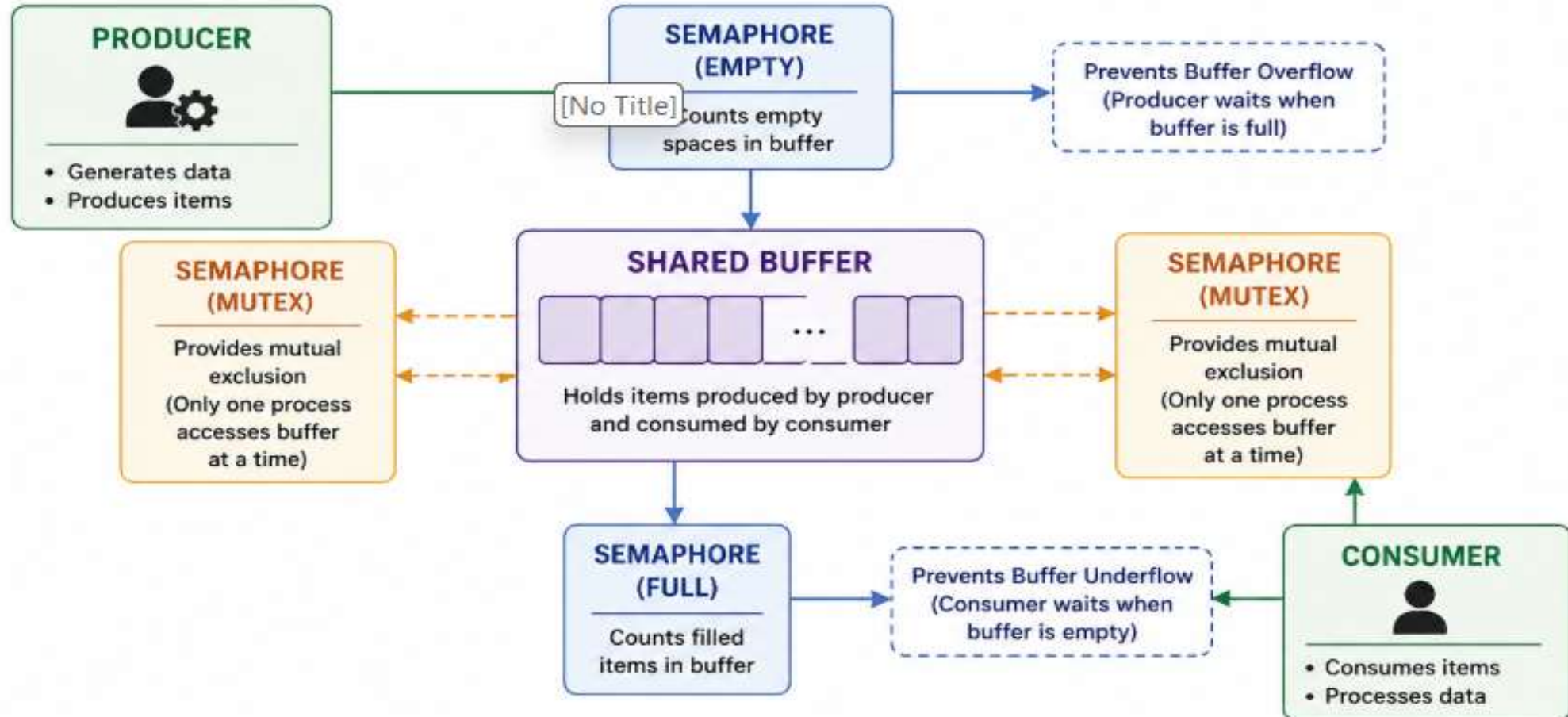
The system is implemented using **C programming on Linux/Ubuntu with POSIX/Linux APIs**.

- In a multitasking Operating System, multiple processes may execute concurrently.
- When multiple processes access the same **shared resource**, synchronization is required.
- The **Producer–Consumer problem** is a classic example of process synchronization.
- The **Producer** generates data and places it into a shared buffer.
- The **Consumer** retrieves data from the buffer and processes it.
- Without proper synchronization, **race conditions, buffer overflow, and buffer underflow** may occur.
- In this project, **semaphores** are used to coordinate the producer and consumer and ensure safe access to the shared buffer

Hardware and Software requirements

HARDWARE REQUIREMENTS	SOFTWARE REQUIREMENTS
• Computer / Laptop	• Linux / Ubuntu
• RAM	• C Programming
• Storage	• GCC Compiler
	• POSIX Semaphores
	• Linux Terminal
	• GitHub

System Architecture



WORKING FLOW:

1. Producer waits on EMPTY semaphore → if space available, it gets the space.
2. Producer waits on MUTEX semaphore → enters critical section and adds item to buffer.
3. Producer signals MUTEX → leaves critical section.
4. Producer signals FULL → notifies consumer that a new item is available.

5. Consumer waits on FULL semaphore → if item available, it gets the item.
6. Consumer waits on MUTEX semaphore → enters critical section and removes item from buffer.
7. Consumer signals MUTEX → leaves critical section.
8. Consumer signals EMPTY → notifies producer that a space is free.

Conclusion

The project demonstrates how **semaphores** can be used to **synchronize concurrent producer and consumer processes** in Linux.

The system ensures:

- Safe access to the shared buffer
- Prevention of race conditions
- Prevention of buffer overflow
- Prevention of buffer underflow
- Concurrent execution of producer and consumer
- Effective management of shared resources

The project provides practical understanding of **concurrency, process synchronization, critical sections, mutual exclusion, shared memory, and semaphores**.

Thank you